```
Admin::User
```

If `User` is missing, in either case all Rails knows is that a constant called "User" was missing in a module called "Admin".

If there is a top-level `User` Ruby would resolve it in the former example, but wouldn't in the latter. In general, Rails does not emulate the Ruby constant resolution algorithms, but in this case it tries using the following heuristic:

If none of the parent namespaces of the class or module has the missing constant then Rails assumes the reference is relative. Otherwise qualified.

For example, if this code triggers autoloading

```
Admin::User
```

and the `User` constant is already present in `Object`, it is not possible that the situation is

```
module Admin
  User
end
```

because otherwise Ruby would have resolved `User` and no autoloading would have been triggered in the first place. Thus, Rails assumes a qualified reference and considers the file `admin/user.rb` and directory `admin/user` to be the only valid options.

In practice, this works quite well as long as the nesting matches all parent namespaces respectively and the constants that make the rule apply are known at that time.

However, autoloading happens on demand. If by chance the top-level `User` was not yet loaded, then Rails assumes a relative reference by contract.

Naming conflicts of this kind are rare in practice, but if one occurs, `require_dependency` provides a solution by ensuring that the constant needed to trigger the heuristic is defined in the conflicting place.

6.3 Automatic Modules

When a module acts as a namespace, Rails does not require the application to defines a file for it, a directory matching the namespace is enough.

Suppose an application has a back office whose controllers are stored in `app/controllers/admin`. If the `Admin` module is not yet loaded when `Admin::UsersController` is hit, Rails needs first to autoload the constant `Admin`.

If `autoload_paths` has a file called `admin.rb` Rails is going to load that one, but if there's no such file and a directory called `admin` is found, Rails creates an empty module and assigns it to the `Admin`constant on the fly.

6.4 Generic Procedure

Relative references are reported to be missing in the cref where they were hit, and qualified references are reported to be missing in their parent (see Resolution Algorithm for Relative Constantsat the beginning of this guide for the definition of *cref*, and Resolution Algorithm for Qualified Constants for the definition of *parent*).

The procedure to autoload constant `C` in an arbitrary situation is as follows:

```
if the class or module in which C is missing is Object
  let ns = ''
else
  let M = the class or module in which C is missing

  if M is anonymous
```